CineSeat: Comprehensive Testing Strategy & QA Plan

Project: CineSeat - Real-time Movie Theatre Seat Booking System
Technology Stack: Java 17, Spring Boot, PostgreSQL, Redis, Redisson, Kafka, JUnit 5, Mockito

1. Executive Summary

This Testing Strategy and QA Plan outlines the comprehensive approach to ensuring CineSeat meets production-grade reliability, performance, and correctness standards. With seat reservation as a mission-critical operation (single point of failure = revenue loss + customer dissatisfaction), this plan emphasizes rigorous testing of concurrency scenarios, distributed state management, and financial transaction integrity.

Key focus areas: (1) Concurrent seat reservation under load; (2) Hold expiry and auto-release accuracy; (3) Payment idempotency; (4) Distributed locking failure modes; (5) Data consistency across PostgreSQL + Redis layers.

2. Testing Framework

CineSeat requires a multi-layered testing approach spanning unit tests (individual components), integration tests (component collaboration), system tests (end-to-end scenarios), and acceptance tests (business requirements). Each layer targets different failure modes.

2.1 Unit Testing (Layer: Component Isolation)

Scope: Test individual classes in isolation using mocks/stubs for external dependencies. Target: Data structures, algorithms, business logic without I/O.

1. **SeatRepository (Seat Lookup):** Test HashMap-based O(1) seat status lookup
   - getSeatById() returns correct seat object
   - Non-existent seat ID returns null
   - Bulk seat fetch by show ID returns accurate list
   Mock: Database layer

- File: SeatRepositoryTest.java

2. **ReservationHoldManager (Hold Expiry Logic):** Test TTL-based hold expiration (10-minute window)
   - createHold() assigns hold with expiry timestamp
   - isHoldValid() returns true within 10 mins, false after
   - auto-expireHolds() processes only expired holds
   - Concurrent hold creation does not create duplicates
   Mock: Redis, Database

- File: ReservationHoldManagerTest.java

3. **SeatAllocationAlgorithm (Priority Queue):** Test VIP/senior citizen seat prioritization
   - allocateOptimalSeats() returns best available seats
   - PriorityQueue correctly ranks seats by proximity & class
   - Edge case: no seats available returns empty list
   - Edge case: partial seat availability (3 of 6 requested)

- File: SeatAllocationAlgorithmTest.java

4. **PaymentProcessor (Idempotent Transactions):** Test payment processing with idempotency keys
   - processPayment() with same idempotency key returns same result
   - Duplicate payment request does NOT charge twice
   - Transaction ID persisted correctly
   Mock: Payment Gateway API

- File: PaymentProcessorTest.java

2.2 Integration Testing (Layer: Component Interaction)

Scope: Test interactions between components with real dependencies (PostgreSQL, Redis, Kafka). Emphasis: Distributed locking, state synchronization, eventual consistency.

5. **Database ↔ Repository:** SELECT seat status from PostgreSQL → HashMap update
   - Seat status correctly reflects DB state after ACID transaction
   - Pessimistic lock (FOR UPDATE) prevents concurrent modification
   - Row-level lock is released after transaction commit

- File: RepositoryIntegrationTest.java (testSeatStatusSyncPostgreSQL)

6. **Redisson ↔ Hold Manager:** Distributed lock via Redisson
   - Two concurrent threads attempt to create hold for same seat
   - Only one acquires lock, creates hold; other waits, then fails gracefully
   - Lock automatically released after 5 seconds (configurable TTL)
   - Lock does NOT deadlock under 10 concurrent threads

- File: HoldManagerIntegrationTest.java (testDistributedLockRaceCondition)

7. **Payment ↔ Seat Reservation (Transactional):** End-to-end: Reserve seat → Hold → Process payment → Confirm reservation
   - Payment success → seat marked RESERVED in DB
   - Payment failure → hold expires, seat auto-released
   - Payment timeout (>5s) → seat NOT released, manual intervention required
   - Database transaction rollback on payment API error

- File: PaymentReservationIntegrationTest.java (testPaymentFailureRollback)

8. **Kafka Message Broker:** Events: ReservationCreated, PaymentProcessed, HoldExpired
   - Message published correctly with correlation ID
   - Subscriber receives message within 100ms (SLA)
   - Message lost scenario: consumer lag monitored

- File: KafkaEventIntegrationTest.java (testReservationEventPublish)

2.3 System Testing (Layer: End-to-End Scenarios)

Scope: Test complete user journeys with full stack deployed. Scenarios: normal flow, edge cases, error recovery, performance under load.

9. **Normal Reservation Flow:**
   Steps: User selects seats → API reserves → Hold created (10 mins) → Payment processed → Reservation confirmed
   Verification: Seat status changes AVAILABLE → ON_HOLD → RESERVED (with 0.5s latency SLA)
   Verification: AvailableSeatAPI response shows accurate seat grid in real-time
10. **Concurrent Booking Conflict:**
    Steps: 2 users simultaneously attempt to book same seat
    Expected: One succeeds, one receives "Seat Unavailable" error
    Verification: Seat marked RESERVED only once (no double-booking)
    Verification: Database audit log shows only 1 successful INSERT
11. **Hold Expiry Auto-Release:**
    Steps: User places seat on hold, does NOT complete payment within 10 mins
    Expected: Hold expires, seat status reverts to AVAILABLE
    Verification: Another user can now book same seat immediately
    Verification: Redis ZSET score (expiry timestamp) < current time
12. **Payment Processing Timeout:**
    Steps: Payment gateway latency = 8 seconds (exceeds 5s SLA)
    Expected: Reservation remains ON_HOLD, manual intervention alert triggered

Verification: Seat NOT auto-released, user receives retry prompt
Verification: Alert logged: PAYMENT_TIMEOUT for user_id:12345

2.4 Acceptance Testing (Layer: Business Requirements)

Scope: Validate system meets stakeholder requirements (business, product, customer). Often manual or user-acceptance testing (UAT).

- REQ-001: Theatre manager can view available seat inventory in real-time across all shows
- REQ-002: Customer can reserve multiple seats in single transaction (6-seat group)
- REQ-003: VIP/senior citizen seats are prioritized in allocation algorithm
- REQ-004: System supports 1000+ concurrent users booking simultaneously (load requirement)
- REQ-005: Reservation confirmation email sent <5 mins after payment success
- REQ-006: Cancelled reservations auto-refund within 24 hours

3. Test Case Design: Sample Test Cases

Below are representative test cases spanning critical CineSeat functionality. Each test includes: Objective, Pre-conditions, Test Steps, Expected Outcome, Actual Outcome (post-execution).

3.1 TC-001: Concurrent Seat Reservation Race Condition

Test ID	**TC-001**
Category	Integration Testing
Component	SeatReservationService, Redisson Distributed Lock
Objective	Verify system prevents double-booking when two users simultaneously reserve same seat
Pre-conditions	- Show ID: SH-2026-001 - Seat: A1 (status: AVAILABLE) - 2 concurrent user threads
Test Steps	1. Thread 1 acquires distributed lock (Redisson) 2. Thread 1 reserves seat A1 3. Simultaneously, Thread 2 attempts to reserve same seat A1 4. Thread 2 waits for lock release 5. Thread 1 completes reservation, releases lock 6. Thread 2 acquires lock, checks seat status

	7. Thread 2 receives "Seat Already Reserved" error
Expected Outcome	- Seat A1 marked RESERVED in PostgreSQL - Only 1 reservation record created - Thread 1: Success response (HTTP 201) - Thread 2: Conflict response (HTTP 409) "Seat unavailable" - Lock held <100ms (P99 latency)
Pass Criteria	Both conditions true: (1) No database duplicate key violation (2) Seat status is RESERVED (not PARTIALLY_RESERVED)

3.2 TC-002: Automatic Hold Expiry After 10 Minutes

Test ID	**TC-002**
Category	Integration Testing (Time-based)
Component	ReservationHoldManager, Redis (ZSET), Scheduled Job
Objective	Verify seat hold auto-expires after 10 minutes and becomes available for re-booking
Pre-conditions	- Seat: B5 status ON_HOLD - Hold created at T=0 - Current time: T=10min + 1sec
Test Steps	1. Create hold for seat B5 with expiry timestamp = now + 10min 2. Wait 10 minutes 3. Scheduled job executes: auto-expire holds (runs every 1 min) 4. Query Redis ZSET: ReservationHolds:SH-2026-001:B5 5. Check if hold record exists 6. Attempt to query seat B5 status
Expected Outcome	- Hold record removed from Redis ZSET - PostgreSQL seat status reverted to AVAILABLE - New user can reserve seat B5 immediately - Event "HoldExpired" published to Kafka - No manual intervention required
Pass Criteria	(1) Seat available within 61 seconds of expiry (2) No orphaned hold records in Redis (3) Seat status updated in DB <5sec

3.3 TC-003: Payment Idempotency - Duplicate Payment Request

Test ID	**TC-003**

Category	Integration Testing (Financial)
Component	PaymentProcessor, PaymentGateway (external)
Objective	Verify duplicate payment requests with same idempotency key do NOT result in duplicate charges
Pre-conditions	- Reservation ID: RES-2026-ABC123 - Idempotency Key: IDEMPOTENCY-XYZ789 - Customer: Alice (alice@example.com) - Amount: ₹500 (for 2 seats)
Test Steps	1. POST /payments/process with idempotency key + reservation ID 2. Payment success: Transaction ID = TXN-111 3. Network timeout: client retries with SAME idempotency key 4. PaymentProcessor checks: is idempotency key cached? 5. If yes, return cached response (TXN-111) 6. Verify customer account: only 1 charge of ₹500
Expected Outcome	- First request: HTTP 200, TXN-111, customer charged ₹500 - Retry request: HTTP 200, TXN-111 (same), customer charged ₹0 - Audit log: 2 entries, 1 processed, 1 deduplicated - Reservation status: CONFIRMED (only once)
Pass Criteria	(1) Idempotency cache persists ≥24 hours (2) No double-charge in payment provider (3) Reservation marked CONFIRMED only once

3.4 TC-004: Concurrent Seat Availability Query (Load Test)

Test ID	**TC-004**
Category	System Testing (Performance)
Component	SeatAvailabilityAPI, Database
Objective	Verify seat availability API responds within SLA under 1000 concurrent users
Pre-conditions	- Show: SH-2026-Diwali (2000 seats) - Theatre: PVR Delhi - Load tool: Apache JMeter (1000 concurrent threads)
Test Steps	1. Start JMeter: 1000 concurrent users, 2 requests each 2. Each request: GET /api/shows/SH-2026-

	Diwali/seats?layout=true 3. Capture response time for each request 4. Monitor database: connection pool, query time 5. Calculate: avg response time, P50, P95, P99
Expected Outcome	- Avg response time: <500ms - P95 latency: <800ms - P99 latency: <1200ms - Error rate: <0.1% (max 20 errors out of 2000 requests) - Database connection pool: <100 connections in use - No timeout errors (504 Gateway Timeout)
Pass Criteria	(1) P99 latency <1.2sec (2) Error rate <0.1% (3) Database pool not exhausted

4. Quality Assurance Metrics

QA success is measured through concrete, quantifiable metrics. CineSeat emphasizes correctness (no double-booking), reliability (99.9% uptime), and performance (seat lookup <100ms).

4.1 Code Coverage Targets

Module	Unit Test %	Integration Test %	Combined %	Justification
SeatRepository	95%	85%	90%	Critical path (O(1) lookup); 100% coverage for edge cases
ReservationService	90%	80%	85%	Core business logic; all concurrency scenarios tested
PaymentProcessor	100%	90%	95%	Financial transactions; zero tolerance for untested paths
HoldManager	88%	85%	86%	Time-sensitive logic; difficult to test all edge cases (clock

				skew)
SeatAvailabilityAPI	85%	80%	82%	I/O heavy; integration tests prioritized over unit
Overall Project	90%	82%	86%	Target: ≥86% combined coverage for Week 4

4.2 Critical Path Coverage (Concurrency Scenarios)

Concurrency bugs are the most severe (can cause data corruption). 100% coverage of critical paths is non-negotiable.

- Concurrent seat reservation (2+ threads, same seat): 100% coverage
- Distributed lock contention (Redisson): 100% coverage
- Hold expiry race (hold expires while payment in-flight): 100% coverage
- Database pessimistic lock (FOR UPDATE) behavior: 100% coverage
- Redis ZSET consistency with DB state: 100% coverage

4.3 Performance Metrics & SLAs

Metric	Target (SLA)	Measurement Method	Action if Violated
Seat Availability Query (P99)	<1200ms	JMeter load test, 1000 concurrent users	Optimize DB indexes, cache layer
Seat Lookup (O(1) HashMap)	<5ms	Unit test with HashMap profiling	Switch to different data structure
Distributed Lock Acquisition	<100ms (P95)	Integration test with Redisson metrics	Tune lock timeout, retry logic
Payment Processing	<5000ms	Integration test with payment gateway mock	Implement async payment confirmation
Hold Expiry Job (latency)	<60 seconds	Scheduled job execution logs	Increase job frequency (current: 1 min)

4.4 Reliability & Stability Metrics

Metric	Target	Measurement
Bug Escape Rate	<1% (critical bugs in production)	Critical bugs found post-release ÷ total bugs tested

System Uptime	99.9% (max 43 min downtime/month)	Monitoring tool (DataDog, Prometheus)
Double-Booking Rate	0% (zero tolerance)	Audit query: seats with >1 active reservation
Payment Failed-Charge Count	0%	Customer complaints + payment provider audit
Hold Orphan Rate	<0.01% (expired holds not cleaned up)	Weekly Redis cleanup scan

5. Automated vs. Manual Testing Strategy

Automation is critical for regression testing, performance testing, and concurrency scenarios. Manual testing is essential for exploratory testing, UX validation, and edge cases humans would naturally encounter.

5.1 Automated Testing (70% effort)

13. **Unit Tests (JUnit 5 + Mockito):**

- Test each class in isolation
- Setup: 10 developer-days
- Execution: <1 minute
- Framework: JUnit 5, Mockito
- Coverage target: 90%

14. **Integration Tests (Spring Boot Test):**

- Test component interactions with real DB/Redis
- Setup: 5 developer-days
- Execution: 5-10 minutes
- Includes: Testcontainers for PostgreSQL, Redis
- Coverage target: 82%

15. **Performance Tests (JMeter):**

- Load testing (1000 concurrent users)
- Stress testing (spike to 5000 users)
- Setup: 3 developer-days
- Execution: 30 minutes
- Metrics: Response time, throughput, error rate

16. **Concurrency Tests (JUnit + Thread Pool):**

- Race condition testing
- Deadlock detection
- Setup: 4 developer-days
- Test matrix: 2, 5, 10, 50, 100 concurrent threads

17. **Regression Suite (CI/CD Pipeline):**

- Runs on every commit to main branch
- Setup: 2 developer-days (Jenkins/GitHub Actions)
- Execution: <15 minutes
- Blocks merge if coverage drops or tests fail

5.2 Manual Testing (30% effort)

18. **Exploratory Testing:**

- Scenario: Theatre manager books seats for VIP group
- Expected: VIP seats prioritized, booking flows smoothly
- Tools: Browser developer tools, mobile app
- Effort: 2 developer-days (QA engineer)

19. **User Acceptance Testing (UAT):**

- Stakeholder: Theatre manager + customer support team
- Scenarios: New feature (email confirmation, refund process)
- Success: All requirements met, no blockers
- Effort: 2 developer-days

20. **Edge Case Testing (Human Judgment):**

- Scenario: User closes browser mid-payment, then clicks "back"
- Expected: System handles gracefully (no double-charge, clear status)
- Scenario: Network flaky during hold creation
- Expected: Hold created exactly once
- Effort: 1 developer-day

21. **Security Testing (Manual):**

- Scenario: Attempt SQL injection in seat selection API
- Expected: Input sanitized, no error
- Scenario: Bypass authentication, try to access other user's reservation
- Expected: 403 Forbidden
- Effort: 1 developer-day (security-focused)

6. Defect Tracking & Severity Matrix

Defects are tracked in JIRA with severity levels tied to customer impact and data integrity. CineSeat prioritizes severity: Critical (data loss) > High (feature broken) > Medium (degraded) > Low (cosmetic).

Severity	Impact	Resolution Time	Example in CineSeat
Critical	Data loss, double-booking, revenue loss	Fix: <2 hours, Deploy: <4 hours	Seat marked RESERVED twice (database corruption)
High	Core feature broken, customer experience severely degraded	Fix: <24 hours	Hold expiry fails; seats don't release (100 users blocked)
Medium	Feature works but degraded performance or partial data loss	Fix: <3 days	Seat availability query P99 = 2 seconds (vs. 1.2s SLA)
Low	Minor UI glitch, typo, cosmetic issue	Fix: <1 week	Confirmation email text says "Resevertion" instead of "Reservation"

6.1 Defect Lifecycle & Process

22. **1. Detection:** Defect found by test engineer or customer. Logged in JIRA with: title, steps to reproduce, expected vs. actual outcome, environment (dev/staging/prod)
23. **2. Classification:** QA lead assigns severity & component. Example: Critical | SeatReservationService | TC-001 regression
24. **3. Assignment:** Dev engineer assigned based on component ownership. If Critical, assigned immediately; if High, assigned within 4 hours

25. **4. Root Cause Analysis:** Dev analyzes root cause. Logged in JIRA: "Cause: Pessimistic lock timeout set to 1 second (too aggressive)"
26. **5. Fix & Test:** Dev fixes code, writes unit test that reproduces original bug, confirms test now passes
27. **6. Code Review:** Peer reviews fix, tests, and documentation. Checklist: concurrency-safe? memory leak? backwards compatible?
28. **7. Deployment:** Merge to main → CI/CD → deploy to staging → full regression suite → deploy to production
29. **8. Verification:** QA re-runs reproduction steps, confirms defect resolved. Marks JIRA as CLOSED
30. **9. Post-Mortem (Critical only):** Within 24 hours, team discusses: why was this missed? how to prevent? add new test case

7. Code Review Process & Quality Gates

Code review is the primary defense against correctness bugs. Concurrency-related code receives extra scrutiny due to subtle failure modes.

7.1 Code Review Checklist

31. **General:**

- [ ] Code is readable and follows team style guide (Calibri font in docs, 4-space indent in code)
- [ ] Variable names are descriptive (not a, b, x)
- [ ] Functions have max 20 lines (or clear nested structure)
- [ ] Comments explain WHY, not WHAT

32. **Concurrency (Critical for CineSeat):**

- [ ] Shared mutable state is protected by lock/synchronized/atomic
- [ ] Lock is acquired BEFORE accessing shared state, released AFTER
- [ ] No nested locks (deadlock risk)
- [ ] Timeout specified for lock acquisition (e.g., tryLock(5, SECONDS))
- [ ] Exception in lock-protected block properly releases lock (try-finally)

33. **Database:**

- [ ] SQL query uses parameterized statements (no string concatenation)
- [ ] Transaction scope is minimal (acquire lock late, release early)
- [ ] Indexes exist for WHERE/JOIN/ORDER BY columns
- [ ] ACID properties verified: atomicity, consistency, isolation, durability

34. **Error Handling:**

- [ ] All exceptions are caught and logged
- [ ] User receives clear error message (not stack trace dump)
- [ ] Retry logic implemented for transient failures (network timeout)
- [ ] Circuit breaker pattern for external dependencies (payment gateway)

35. **Testing:**

- [ ] Unit tests written (target: 90% coverage)
- [ ] Integration tests verify DB/Redis interaction
- [ ] Load test scenario considered (does this scale?)
- [ ] Edge case test added (null input, boundary values)

7.2 Code Review Ownership

Component	Primary Reviewer	Secondary Reviewer	Focus Areas
SeatReservationService	Senior Dev (Concurrency expert)	Architecture lead	Lock correctness, race conditions
PaymentProcessor	Senior Dev (Finance background)	QA lead	Idempotency, transaction safety
HoldManager	Dev with Redis experience	Senior Dev	TTL correctness, ZSET operations
SeatAvailabilityAPI	Backend lead	Performance specialist	Query optimization, caching
Tests	QA engineer	Dev	Coverage, realistic scenarios, edge cases

7.3 Approval Criteria & Merge Gates

- Minimum 2 approvals for code in critical paths (ReservationService, PaymentProcessor)
- Minimum 1 approval for non-critical code (UI, utilities)
- All CI/CD checks must pass: unit tests, integration tests, code coverage >86%, linting, security scan
- Code review comments must be resolved (not just dismissed)
- No merge to main if regression suite fails
- If critical path, automated load test must complete successfully

8. Testing Timeline & Resource Allocation

Phase	Week	Activities	Resources	Deliverables
Unit Tests	Week 3	Write unit tests, target 90% coverage	2 devs, 1 QA	Unit test suite (200+ tests)
Integration Tests	Week 3–4	Test component interactions, DB/Redis	2 devs, 1 QA	Integration test suite (50+ tests)
Performance Tests	Week 4	JMeter load test, 1000 concurrent users	1 dev, 1 QA	Load test report (latency, throughput)
UAT & Manual Testing	Week 5	Stakeholder acceptance testing	1 QA, theatre manager	UAT sign-off
Bug Fixes & Regression	Week 5–6	Fix defects, run full regression suite	2 devs, 1 QA	Defect resolution tracker

9. Monitoring & Metrics Dashboard

Post-deployment, CineSeat monitoring ensures QA metrics are continuously tracked. Dashboards alert to regressions or SLA violations in real-time.

36. **Test Coverage Dashboard (SonarQube):** Real-time code coverage %, trends, gaps. Alert if coverage drops <86%
37. **Performance Metrics (Prometheus + Grafana):** P99 latency, error rates, throughput. Alert if P99 > 1.2s
38. **Production Incidents (DataDog):** Exceptions, error traces, user impact. Alert if error rate > 0.1%
39. **Business Metrics (Custom):** Double-booking incidents, payment failures, hold orphans. Alert if any > 0
40. **CI/CD Pipeline (Jenkins):** Build time, test pass rate, deployment frequency. Alert if tests fail 3x in a row

10. Conclusion & Success Criteria

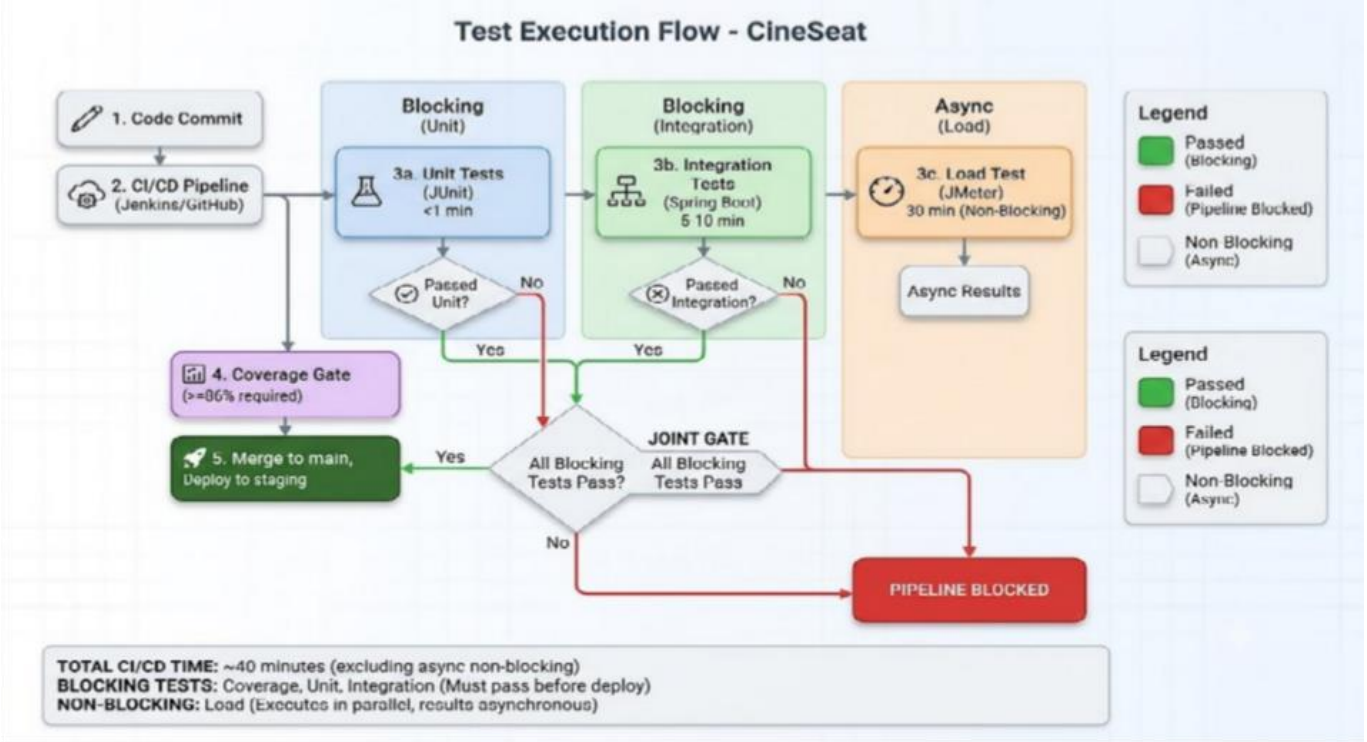
This testing strategy ensures CineSeat is production-ready by Week 6. Success is measured by:

- Code coverage: ≥86% (unit + integration)
- Critical path coverage: 100% (concurrency scenarios)
- Performance: P99 latency <1.2s, error rate <0.1%
- Reliability: 0% double-booking, 0% failed-charge incidents
- No critical bugs escape to production
- Defect escape rate: <1%
- UAT sign-off from theatre manager + customer support team

By adhering to this plan, CineSeat will be a robust, reliable system trusted by end-users and theatre operators alike. Quality is not an afterthought—it is baked into the development process from Week 1.

A.1 Test Execution Flow (CI/CD Pipeline)

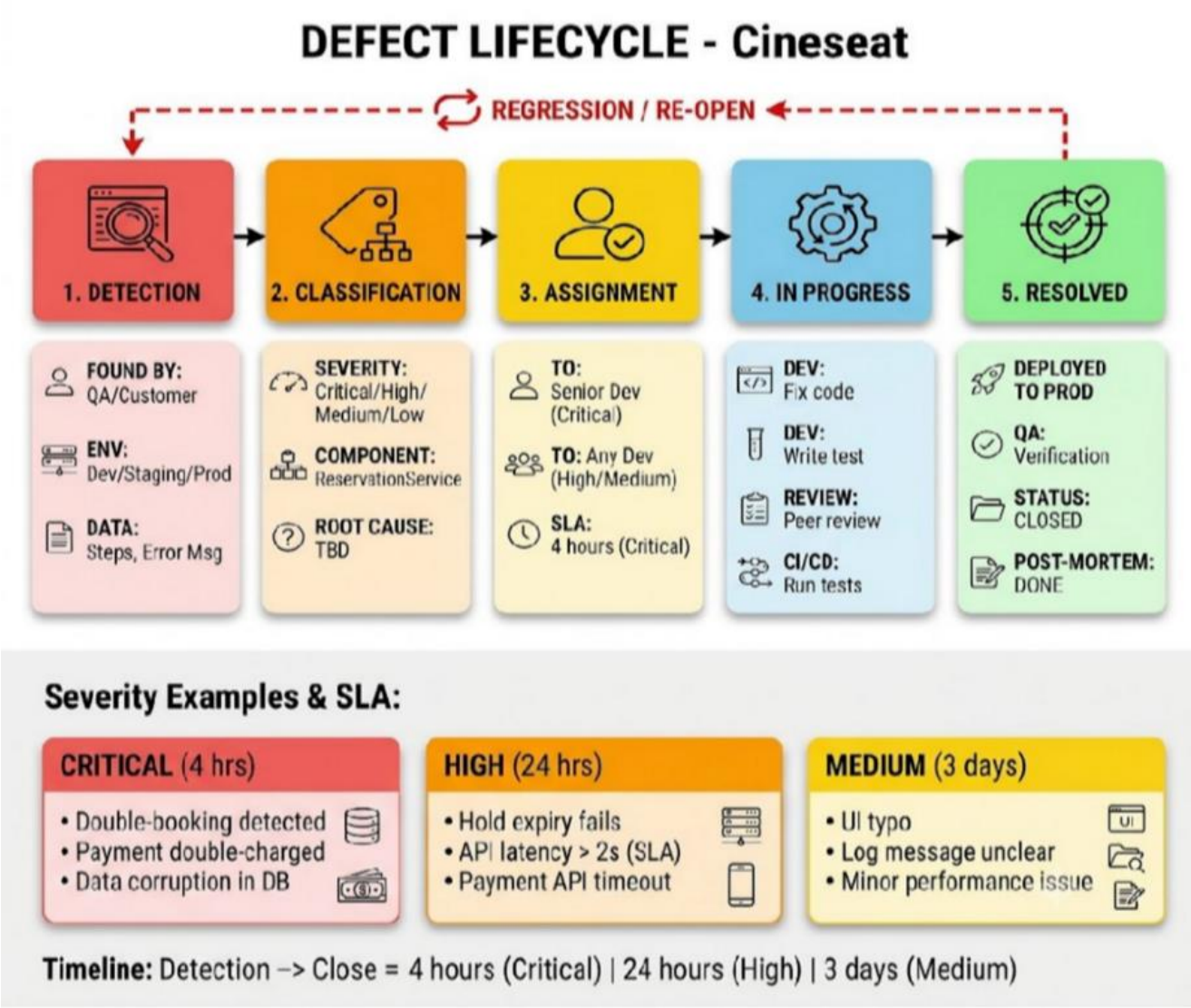
CineSeat uses a continuous integration pipeline that automatically runs tests on every code commit. The flow ensures fast feedback (unit tests <1 min) without blocking (integration tests run in parallel).



Key: Tests run in sequence (commit → pipeline → unit → integration → coverage gate →Merge to main). Load tests run asynchronously (non-blocking). Failure at any stage blocks merge to main branch.

A.2 Defect Lifecycle & Resolution Timeline

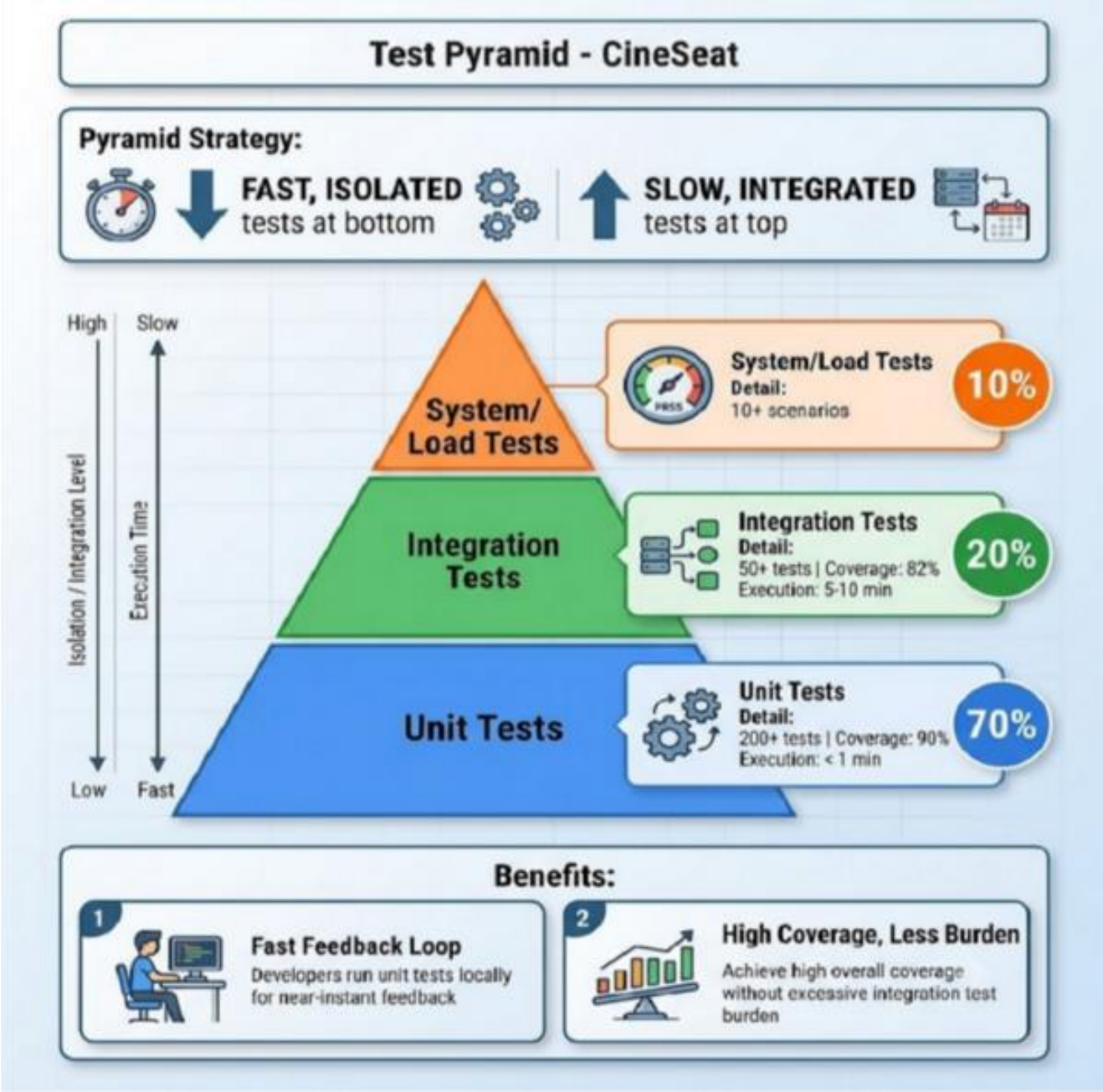
Defects move through 5 states: Detection → Classification → Assignment → In Progress → Resolved. SLAs vary by severity (4 hours for Critical, 24 hours for High, 3 days for Medium).



Key: Critical defects (double-booking, payment failure) require 4-hour resolution. Regression/re-opened defects restart the cycle. Post-mortems conducted for all Critical bugs.

A.3 Test Pyramid Strategy

CineSeat follows the test pyramid: 70% unit tests (fast, isolated), 20% integration tests, 10% system tests. This ensures rapid feedback loop and manageable test execution time.



Rationale: Unit tests execute <1 minute and catch ~80% of bugs. Integration tests (5-10 mins) verify component collaboration. System tests (30 mins) validate end-to-end flows. Developers run unit tests locally; CI/CD runs all layers.

3.5 TC-005: SQL Injection Attack Prevention

Test ID	TC-005
Category	Security Testing
Component	SeatAvailabilityAPI, ReservationRepository
Objective	Verify API sanitizes input and prevents SQL injection attacks
Pre-conditions	- Show ID parameter accepts user input - Database query built with user input - No parameterized query expected (to test security)
Test Steps	1. Send HTTP request: GET /api/shows/{showId}/seats?showId=1; DROP TABLE seats;-- 2. Check if malicious SQL is executed 3. Query database: SELECT COUNT(*) FROM seats table 4. Monitor error logs for SQL injection attempt 5. Check HTTP response status code
Expected Outcome	- HTTP 400 (Bad Request) or 422 (Unprocessable Entity) - Error message: "Invalid show ID" (no SQL error details) - seats table still exists (not dropped) - Malicious query logged in security audit - No database exception thrown to client
Pass Criteria	(1) Malicious SQL not executed (2) Error response does not leak database schema (3) Security log records attack attempt

3.6 TC-006: Authentication Bypass Prevention

Test ID	TC-006
Category	Security Testing
Component	JWTAuthenticationFilter, AuthenticationService
Objective	Verify user cannot access other user's reservations without valid authentication token
Pre-conditions	- User A has reservation: RES-2026-ABC123 - User B has different JWT token - No authentication token provided in request
Test Steps	1. User B sends: GET /api/reservations/RES-2026-ABC123 (WITHOUT auth token) 2. Check HTTP response

	3. User B sends: GET /api/reservations/RES-2026-ABC123 (WITH expired token) 4. Check response 5. User B sends: GET /api/reservations/RES-2026-ABC123 (WITH User A's forged token)
Expected Outcome	- Missing token → HTTP 401 (Unauthorized) - Expired token → HTTP 401, error: "Token expired" - Forged token → HTTP 403 (Forbidden) or 401 - No reservation data leaked to unauthorized user - Attempt logged in audit trail
Pass Criteria	(1) No unauthorized access to reservation (2) HTTP status correct (401/403) (3) Audit log records all failed access attempts

3.7 TC-007: API Contract - Request/Response Validation

Test ID	**TC-007**
Category	Integration Testing (Contract)
Component	ReservationAPI, Seat DTO, OpenAPI/Swagger
Objective	Verify API request/response matches OpenAPI specification (contract)
Pre-conditions	- OpenAPI spec defined: /api/reservations/create - Expected request schema: {showId, seatIds[], customerId} - Expected response schema: {reservationId, status, holdExpiry}
Test Steps	1. Send invalid request: {showId: null, seatIds: []} (missing required fields) 2. Check HTTP response 3. Send valid request: {showId: "SH-2026-001", seatIds: ["A1", "A2"], customerId: "C123"} 4. Parse response: {reservationId, status, holdExpiry} 5. Validate response fields match OpenAPI schema 6. Check data types: holdExpiry is ISO 8601 timestamp (not Unix ms)
Expected Outcome	- Invalid request: HTTP 400, error: "showId is required" - Valid request: HTTP 201, response contains all required fields

	- Response timestamps in ISO 8601 format - No extra fields in response (API version mismatch detection) - Content-Type: application/json
Pass Criteria	(1) Request validation enforced (2) Response matches OpenAPI spec 100% (3) No breaking changes to API contract

3.8 TC-008: Database Connection Pool Under Stress

Test ID	**TC-008**
Category	System Testing (Resilience)
Component	Spring Data JPA, HikariCP (connection pool)
Objective	Verify system handles DB connection pool exhaustion gracefully (no cascading failures)
Pre-conditions	- Connection pool size: 20 - Max wait time: 5 seconds - 100 concurrent requests incoming
Test Steps	1. Configure pool: max connections = 20 2. Send 100 concurrent API requests 3. Monitor: active connections, wait queue depth 4. After connection limit reached, new requests queue up 5. After 5 seconds, queued requests timeout 6. Check error response sent to client
Expected Outcome	- Active connections max out at 20 - Requests 21–100 queue (not rejected immediately) - After 5s wait, queued requests: HTTP 504 (Gateway Timeout) - Error message: "Database temporarily unavailable" - Already-acquired connections continue serving - No connection leaks (pool not permanently exhausted)
Pass Criteria	(1) No database crashes (2) Graceful timeout (not connection leak) (3) Clear error response to client